Cross Coverage & SVA Assignment

Q1.

```
covergroup cvr_grp;
    //ALSU_2 ALSU_3
    A_cvp_values_ADD_MULT : coverpoint A {
        bins A_data_0 = {0};
        bins A_data_max = {MAXPOS};
        bins A_data_min = {MAXNEG};
        bins A_data_default = default;
    }
    //ALSU_5 ALSU_7
    A_cvp_values_RED : coverpoint A iff (red_op_A) {
        bins A_walkingones[] = {3'b001, 3'b010, 3'b100};
    }
    //ALSU_2 ALSU_3
    B_cvp_values_ADD_MULT : coverpoint B {
        bins B_data_0 = {0};
        bins B_data_max = {MAXPOS};
        bins B_data_min = {MAXNEG};
        bins B_data_default = default;
    }
    //ALSU_5 ALSU_7
    B_cvp_values_RED : coverpoint B iff (red_op_B && !red_op_A) {
        bins B_walkingones[] = {3'b001, 3'b010, 3'b100};
    }
```

```
//ALSU_8 ALSU_9 ALSU_10 ALSU_12
opcode_cvp_values : coverpoint opcode {
    bins bins_shift_rot[] = {SHIFT, ROTATE};
    bins bins_arith[] = {ADD, MULT};
    illegal_bins opcode_invalid = {INVALID_6, INVALID_7};
    bins opcode_valid_trans = (OR => XOR => ADD => MULT => SHIFT => ROTATE);
}

opcode_bitwise_cp: coverpoint opcode{
    bins bins_bitwise[] = {OR, XOR};
}
//ALSU_2
cross_ARITH_PERM: cross A_cvp_values_ADD_MULT, B_cvp_values_ADD_MULT, opcode_cvp_values {
    ignore_bins ig_bins_shift = binsof(opcode_cvp_values.bins_shift_rot);
    ignore_bins ig_bins_trans = binsof(opcode_cvp_values.opcode_valid_trans);
}
//ALSU_2
cross_ARITH_CIN: cross cin, opcode_cvp_values {
    option.cross_auto_bin_max = 0;
    bins ig_bins_shift = binsof(opcode_cvp_values.bins_arith);
}
//ALSU_8
cross_SHIFT_opcode: cross direction, opcode_cvp_values {
    option.cross_auto_bin_max = 0;
    bins ig_bins_arith = binsof(opcode_cvp_values.bins_shift_rot);
}
```

Kareem Waseem

```
opcode_SHIFT_cp: coverpoint opcode {
   bins bins_SHIFT = {SHIFT};
}
//ALSU_8
cross_SHIFT: cross opcode_SHIFT_cp, serial_in;
//ALSU_5
cross_reduction_A: cross A_cvp_values_RED, B_cvp_values_ADD_MULT, opcode_bitwise_cp iff (red_op_A)
{
   ignore_bins B_data_max_ = binsof(B_cvp_values_ADD_MULT.B_data_max);
   ignore_bins B_data_min_ = binsof(B_cvp_values_ADD_MULT.B_data_min);
}
//ALSU_5
cross_reduction_B: cross B_cvp_values_RED, A_cvp_values_ADD_MULT, opcode_bitwise_cp iff (red_op_B)
{
   ignore_bins A_data_max_ = binsof(A_cvp_values_ADD_MULT.A_data_max);
   ignore_bins A_data_min_ = binsof(A_cvp_values_ADD_MULT.A_data_min);
}

opcode_not_bitwise_cp: coverpoint opcode {
   option.weight = 0;
   bins bins_not_bitwise[] = {[ADD:$]};
}
//ALSU 10
cross_invalid_A: cross red_op_A, opcode_not_bitwise_cp {
   ignore_bins red_A_0 = binsof(red_op_A) intersect {0};
   illegal_bins red_A_0_ill = binsof(red_op_A) intersect {1};
}
```

```
   //ALSU 10
   cross_invalid_B: cross red_op_B, opcode_not_bitwise_cp {
      ignore_bins red_B_0 = binsof(red_op_B) intersect {0};
      illegal_bins red_B_0_ill = binsof(red_op_B) intersect {1};
   }

endgroup
```

Q2.

1- assert property (@(posedge clk) a |-> ##2 b);

2- assert property (@(posedge clk) (a && b) |-> ##[1:3] c);

3- sequence s11b;

##2 ~b;

endsequence

4- property p_decoder;

@(posedge clk) $onehot(Y);

endproperty

5- property p_valid;

@(posedge clk) ($countones(D) == 0) |=> ~valid;

Endproperty

Kareem Waseem

Q3.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
COUNTER_1	When the reset is asserted, the output counter value should be low	Directed at the start of the sim, then randomized with constraint that drive the reset to be off most of the simulation time.	-	Immediate assertion to check for the async reset functionality
COUNTER_2	When the load is asserted, the output count_out should take the value of the load_data input	Randomization under constraints on the load signal to be off 70% of the time	Cover all values of load data	Concurrent assertion to check the load_data
COUNTER_3	When the load/reset is deasserted, the output count_out increment as long as the enable is active and up_down is high	Randomization for up_down, and randomization for enable to be 70% of the time	Cover all values of count_out, and transition bin from max to zero	Concurrent assertion to check for the counter increment
COUNTER_4	When the load/reset is deasserted, the output count_out decrement as long as the enable is active and up_down is low	Randomization for up_down, and randomization for enable to be 70% of the time	Cover all values of count_out, and transition bin from zero to max	Concurrent assertion to check for the counter decrement
COUNTER_5	When the reset/load control signal is not active, and the enable is off then the dout does not change	-	-	Concurrent assertion to check for the counter output does not change
COUNTER_6	When the count_out is 0 then the zero output is high	-	-	Concurrent assertion to check for the zero output once the counter output is 0
COUNTER_7	When the count_out is 0 then the max_count output is high	-	-	Concurrent assertion to check for the max_count output once the counter output is maximum

```
module counter (counter_if.DUT c_if);

always @(posedge c_if.clk or negedge c_if.rst_n) begin
    if (!c_if.rst_n)
        c_if.count_out <= 0;
    else if (!c_if.load_n)
        c_if.count_out <= c_if.data_load;
    else if (c_if.ce)
        if (c_if.up_down)
            c_if.count_out <= c_if.count_out + 1;
        else
            c_if.count_out <= c_if.count_out - 1;
end

assign c_if.max_count = (c_if.count_out == {c_if.WIDTH{1'b1}})? 1:0;
assign c_if.zero = (c_if.count_out == 0)? 1:0;

endmodule
```

```
interface counter_if(clk);
parameter WIDTH = 4;
input clk;
logic rst_n;
logic load_n;
logic up_down;
logic ce;
logic [WIDTH-1:0] data_load;
logic [WIDTH-1:0] count_out;
logic max_count;
logic zero;

modport DUT(input clk, rst_n, load_n, up_down, ce, data_load, output count_out, max_count, zero);

modport TEST (input clk, count_out, max_count, zero, output rst_n, load_n, up_down, ce, data_load);

endinterface
```

Kareem Waseem

```
package counter_pkg;
parameter W = 4;

class counter_transaction;
rand bit rst_n, load_n, up_down, ce;
rand bit [W-1:0] data_load, count_out;

constraint rst_con {
 rst_n dist {0:= 2, 1:= 98};
}

constraint load_con {
 load_n dist {0:= 70, 1:= 30};
}

constraint ce_con {
 ce dist {1:= 70, 0:= 30};
}

covergroup cg();
   load_n_cp: coverpoint load_n
   {
      option.weight = 0;
   }

   data_load_cp: coverpoint data_load
   {
      option.weight = 0;
   }
   //COUNTER_2
   load_cross: cross load_n_cp, data_load_cp {
      ignore_bins bins_load_n_high = binsof(load_n_cp) intersect {1};
   }
   //COUNTER_3
   count_bins_up_cp: coverpoint count_out iff (rst_n && ce && up_down);
   //COUNTER_3
   count_bins_up_trans_cp: coverpoint count_out iff (rst_n && ce && up_down) {
      bins max_zero_trans = ({W{1'b1}} => 0);
   }
```

```
   //COUNTER_4
   count_bins_dn_cp: coverpoint count_out iff (rst_n && ce && !up_down);
   //COUNTER_4
   count_bins_dn_trans_cp: coverpoint count_out iff (rst_n && ce && !up_down) {
      bins zero_max_trans = (0 => {W{1'b1}});
   }
endgroup : cg

function new();
   cg = new();
endfunction

endclass

endpackage
```

```
module counter_sva (counter_if.DUT c_if);

always_comb begin
  if(!c_if.rst_n)
    assert_reset: assert final(c_if.count_out == 0);
end

property count_out_stable;
  @(posedge c_if.clk) disable iff (!c_if.rst_n) (c_if.load_n && !c_if.ce) |=> (c_if.count_out == $past(c_if.count_out));
endproperty

property load_cnt;
  @(posedge c_if.clk) disable iff (!c_if.rst_n) (!c_if.load_n) |=> (c_if.count_out == $past(c_if.data_load));
endproperty

property inc_cnt;
  @(posedge c_if.clk) disable iff (!c_if.rst_n) (c_if.load_n && c_if.ce && c_if.up_down) |=>  (c_if.count_out == ($past(c_if.count_out) + 1'd1));
endproperty

property dec_cnt;
  @(posedge c_if.clk) disable iff (!c_if.rst_n) (c_if.load_n && c_if.ce && !c_if.up_down) |=>  (c_if.count_out == ($past(c_if.count_out) - 1'd1));
endproperty

property zero_cnt;
   @(posedge c_if.clk) disable iff (!c_if.rst_n) (c_if.count_out == {c_if.WIDTH{1'b0}}) |->  (c_if.zero == 1'b1);
endproperty

property max_cnt;
  @(posedge c_if.clk) disable iff (!c_if.rst_n) (c_if.count_out == {c_if.WIDTH{1'b1}}) |->  (c_if.max_count == 1'b1);
endproperty

assert_count_out_stable: assert property (count_out_stable);
assert_load_cnt: assert property (load_cnt);
assert_inc_cnt: assert property (inc_cnt);
assert_dec_cnt: assert property (dec_cnt);
assert_zero_cnt: assert property (zero_cnt);
assert_max_cnt: assert property (max_cnt);

cover_count_out_stable: cover property (count_out_stable);
cover_load_cnt: cover property (load_cnt);
cover_inc_cnt: cover property (inc_cnt);
cover_dec_cnt: cover property (dec_cnt);
cover_zero_cnt: cover property (zero_cnt);
cover_max_cnt: cover property (max_cnt);
endmodule
```

Kareem Waseem

```
import counter_pkg::*;

module counter_tb(counter_if.TEST c_if);

counter_transaction c_txn = new;

// Stimulus Generation
initial begin
    assert_reset;

    repeat(10000) begin
        assert(c_txn.randomize());
        c_if.rst_n = c_txn.rst_n;
        c_if.load_n = c_txn.load_n;
        c_if.up_down = c_txn.up_down;
        c_if.ce = c_txn.ce;
        c_if.data_load = c_txn.data_load;
        @(negedge c_if.clk);
        c_txn.count_out = c_if.count_out;
        c_txn.cg.sample();
    end
    $stop;
end

task assert_reset;
    //assert reset
    c_if.rst_n = 0;
    @(negedge c_if.clk);
    //desert reset
    c_if.rst_n = 1;
endtask

endmodule
```

```
module counter_top();
bit clk;

initial begin
    forever
        #1 clk = ~clk;
end

counter_if c_if(clk);
counter c1(c_if);
counter_tb t1(c_if);
bind counter counter_sva counter_sva_inst(c_if);

endmodule
```

Q4.

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
CONFIG_1	When the reset is asserted, the reset value of the specificed register will be available on the output	Directed at the start of the sim, and throughout the simulation again	-	A checker in the testbench to make sure the output is correct
CONFIG_2	Write FFFF to all registers and read them	Directed to all registers	-	A checker in the testbench to make sure the output is correct
CONFIG_3	Write walking ones to the registers and read them	Directed to all registers	-	A checker in the testbench to make sure the output is correct

```
module test;
 bit clk;
 bit reset;
 bit write;
 bit [15:0] data_in;
 bit [2:0]  address;
 logic [15:0] data_out;

 // Create an associative array to hold the reset values
 bit [15:0]  reset_assoc[string];

 typedef enum {
 ADC0_REG,
 ADC1_REG,
 TEMP_SENSOR0_REG,
 TEMP_SENSOR1_REG,
 ANALOG_TEST,
 DIGITAL_TEST,
 AMP_GAIN,
 DIGITAL_CONFIG
reg_t;

 reg_t my_reg;
 int error = 0, correct = 0;

 // Instantiate config_reg
config_reg config_reg(
    .clk(clk),
    .reset(reset),
    .write(write),
    .data_in(data_in),
    .address(address),
    .data_out(data_out)
 );

 // Create a clock
 initial begin
    forever #1 clk=~clk;
 end
```

```
initial begin
   // Fill the associative array with the expected reset values
   reset_assoc["ADC0_REG"] = 16'hFFFF;
   reset_assoc["ADC1_REG"] = 16'h0;
   reset_assoc["TEMP_SENSOR0_REG"] = 16'h0;
   reset_assoc["TEMP_SENSOR1_REG"] = 16'h0;
   reset_assoc["ANALOG_TEST"] = 16'hABCD;
   reset_assoc["DIGITAL_TEST"] = 16'h0;
   reset_assoc["AMP_GAIN"] = 16'h0;
   reset_assoc["DIGITAL_CONFIG"] = 16'h1;

   //CONFIG_1
   assert_check_reset();

   //CONFIG_2
   write = 1;
   // Write FFFF to every register
   $display("%0t: Writing 16'hFFFF to every register", $time);
   for (int i=0;i<my_reg.last+1;i++) begin
      address = my_reg;
      data_in = 16'hFFFF;
      @(negedge clk);
      my_reg = my_reg.next;
   end

   write = 0;
   // Read every register, expecting FFFF
   $display("%0t: Reading 16'hFFFF from every register", $time);
   for (int i=0;i<my_reg.last+1;i++) begin
      address = my_reg;
      Check16Bits(my_reg.name, 16'hFFFF, data_out);
      my_reg = my_reg.next;
   end

   //CONFIG_1
   assert_check_reset();
```

Karim Waseem

```
   //CONFIG_3
   // Write and read a walking 1's pattern to each register
   $display("%0t: Writing/reading walking 1's to every register", $time);
   for (int i=0;i<my_reg.last+1;i++) begin
      for (int j=0;j<16;j++) begin
         // Do the write
         address = my_reg;
         data_in = 0;
         data_in[j]=1;
         write = 1;
         // Do the read
         @(negedge clk);
         write = 0;
         Check16Bits(my_reg.name, data_in, data_out);
         my_reg = my_reg.next;
      end // Walking 1's
   end // All regs

   $display("%0t: At end of test error = %0d and correct = %0d", $time, error, correct);
   $finish;
end // initial begin

task Check16Bits(input string checked, input bit [15:0] expected, input logic [15:0] actual);
   @(negedge clk);
   if (expected !== actual) begin
      $display("%0t: Read from register %s expected 0x%0h but got 0x%0h", $time, checked, expected, actual);
      error++;
   end
   else
      correct++;
endtask // Check9Bits

task assert_check_reset;
   //assert reset
   reset = 1;
   $display("%0t: Checking reset values", $time);
   for (int i=0;i<my_reg.last+1;i++) begin
      address = my_reg;
      Check16Bits(my_reg.name, reset_assoc[my_reg.name], data_out);
      my_reg = my_reg.next;
   end
   //desert reset
   reset = 0;
endtask
endmodule // test
```

Kareem Waseem

1. *Complete bug report for the 8 bugs*

   a. *adc0_reg:*
      i. *Design Input for Bug to Appear: Writing/reading walking 1's.*
      ii. *Expected Behavior: Read 0001*
      iii. *Observed Behavior: Read 8001*
   b. *adc1_reg:*
      i. *Design Input for Bug to Appear: Writing/reading walking 1's.*
      ii. *Expected Behavior: Read 0001*
      iii. *Observed Behavior: Read 0100*
   c. *temp_sensor0_reg:*
      i. *Design Input for Bug to Appear: Write FFFF to all registers. Then read*
      ii. *Expected Behavior: Read FFFF*
      iii. *Observed Behavior: Read FFFE*
   d. *temp_sensor1_reg:*
      i. *Design Input for Bug to Appear: Write FFFF to all registers. Reset*
      ii. *Expected Behavior: Read 0000*
      iii. *Observed Behavior: Read FFFF*
   e. *analog_test:*
      i. *Design Input for Bug to Appear: Reset*
      ii. *Expected Behavior: Read ABCD*
      iii. *Observed Behavior: Read ABCC*

f. *digital_test:*
    i. *Design Input for Bug to Appear: Writing/reading walking 1's.*
    ii. *Expected Behavior: Read 0001*
    iii. *Observed Behavior: Read 0000*
g. *amp_gain:*
    i. *Design Input for Bug to Appear: Writing/reading walking 1's.*
    ii. *Expected Behavior: Read 0001 before writing the datain value*
    iii. *Observed Behavior: Read 0000 before writing the data in as per the reset value*
h. *digital_config:*
    i. *Design Input for Bug to Appear: Write FFFF to all registers. Then read*
    ii. *Expected Behavior: Read FFFF*
    iii. *Observed Behavior: Read 7FFF*

Kareem Waseem